

DESIGN AND ANALYSIS OF ALGORITHMS

Analysis Framework

Slides courtesy of **Anany Levitin**

Vandana M L

Department of Computer Science & Engineering

What do you mean by analysing an algorithm?

Investigation of Algorithm's efficiency with respect to two resources

- Time
- Space

What is the need for Analysing an algorithm?

- To determine resource consumption
 - CPU time
 - Memory space
- Compare different methods for solving the same problem before actually implementing them and running the programs.
- To find an efficient algorithm

- A measure of the performance of an algorithm
- An algorithm's performance depends on
 - *internal factors*
 - Time required to run
 - Space (memory storage) required to run
 - *external factors*
 - Speed of the computer on which it is run
 - Quality of the compiler
 - Size of the input to the algorithm

important Criteria for performance:

- Space efficiency - the memory required, also called, space complexity
- Time efficiency - the time required, also called time complexity

$$S(P) = C + SP(I)$$

□ Fixed Space Requirements (C)

Independent of the characteristics of the inputs and outputs

- instruction space
- space for simple variables, fixed-size structured variable, constants

□ Variable Space Requirements (SP(I))

dependent on the instance characteristic I

- number, size, values of inputs and outputs associated with I
- recursive stack space, formal parameters, local variables, return address

Algorithm RSUM(A, n)

Input: Array A of numeric values, integer $n \geq 0$

Output: Sum of the n elements of A

if $n = 0$ then

 return 0

else

 return RSUM(A, $n - 1$) + A[$n - 1$]

Each recursive stack frame would contain the following

Reference to an Array (Pointer), Value of n , Return Address,
Temporary space for the returned Value

$$T(P) = C + T_p(I)$$

- Compile time (C)
independent of instance characteristics
- run (execution) time T_p

Design and Analysis of Algorithms

Time Complexity



How to measure time complexity?

- Theoretical Analysis
- Experimental study

Experimental study

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Get an accurate measure of the actual running time
- Use a method like `System.currentTimeMillis()`
- Plot the results

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used
- Experimental data though important is not sufficient

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n .
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Two approaches:

1. Order of magnitude/asymptotic categorization –

This uses coarse categories and gives a general idea of performance.

If algorithms fall into the same category, if data size is small, or if performance is critical, use method 2

2. Estimation of running time -

1. *operation counts* - select operation(s) that are executed most frequently and determine how many times each is done.
2. *step counts* - determine the total number of steps, possibly lines of code, executed by the program.

Design and Analysis of Algorithms

Analysis Framework



- Measuring an input's size
- Measuring running time
- Orders of growth (of the algorithm's efficiency function)
- Worst-base, best-case and average efficiency

Efficiency is defined as a function of input size.

Input size depends on the problem.

Example 1, what is the input size of the problem of sorting n numbers?

Example 2, what is the input size of adding two n by n matrices?

- Measure the running time using standard unit of time measurements, such as seconds, minutes?
Depends on the speed of the computer.
- count the number of times each of an algorithm's operations is executed.
(step count method)
Difficult and unnecessary
- count the number of times an algorithm's basic operation is executed.

Basic operation: the most important operation of the algorithm, the operation contributing the most to the total running time.

For example, the basic operation is usually the most time-consuming operation in the algorithm's innermost loop.

Analysis in the RAM Model

SmartFibonacci(n)	<i>cost</i>	<i>times</i> ($n > 1$)
1 if $n = 0$	c_1	1
2 then return 0	c_2	0
3 elseif $n = 1$	c_3	1
4 then return 1	c_4	0
5 else $pprev \leftarrow 0$	c_5	1
6 $prev \leftarrow 1$	c_6	1
7 for $i \leftarrow 2$ to n	c_7	n
8 do $f \leftarrow prev + pprev$	c_8	$n - 1$
9 $pprev \leftarrow prev$	c_9	$n - 1$
10 $prev \leftarrow f$	c_{10}	$n - 1$
11 return f	c_{11}	1

$$T(n) = c_1 + c_3 + c_5 + c_6 + c_{11} + nc_7 + (n - 1)(c_8 + c_9 + c_{10})$$

$$T(n) = nC_1 + C_2 \Rightarrow T(n) \text{ is a linear function of } n$$

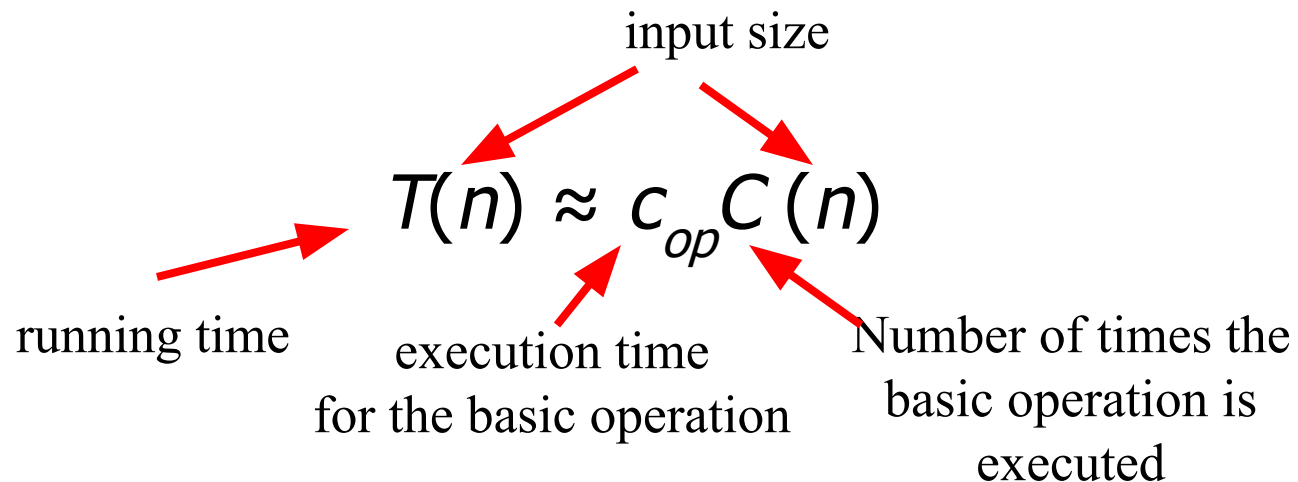
Input Size and Basic Operation Examples

<i>Problem</i>	<i>Input size measure</i>	<i>Basic operation</i>
Search for a key in a list of n items	Number of items in list, n	Key comparison
Add two n by n matrices	Dimensions of matrices, n	addition
multiply two matrices	Dimensions of matrices, n	multiplication

Theoretical Analysis of Time Efficiency: Basic operation count

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size.

Assuming $C(n) = (1/2)n(n-1)$,
how much longer will the algorithm run if we double the input size?



$C(n)$ Basic Operation Count

- The efficiency analysis framework ignores the multiplicative constants of $C(n)$ and focuses on the orders of growth of the $C(n)$.
- Simple characterization of the algorithm's efficiency by identifying relatively significant term in the $C(n)$.

Why do we care about the order of growth of an algorithm's efficiency function, i.e., the total number of basic operations?

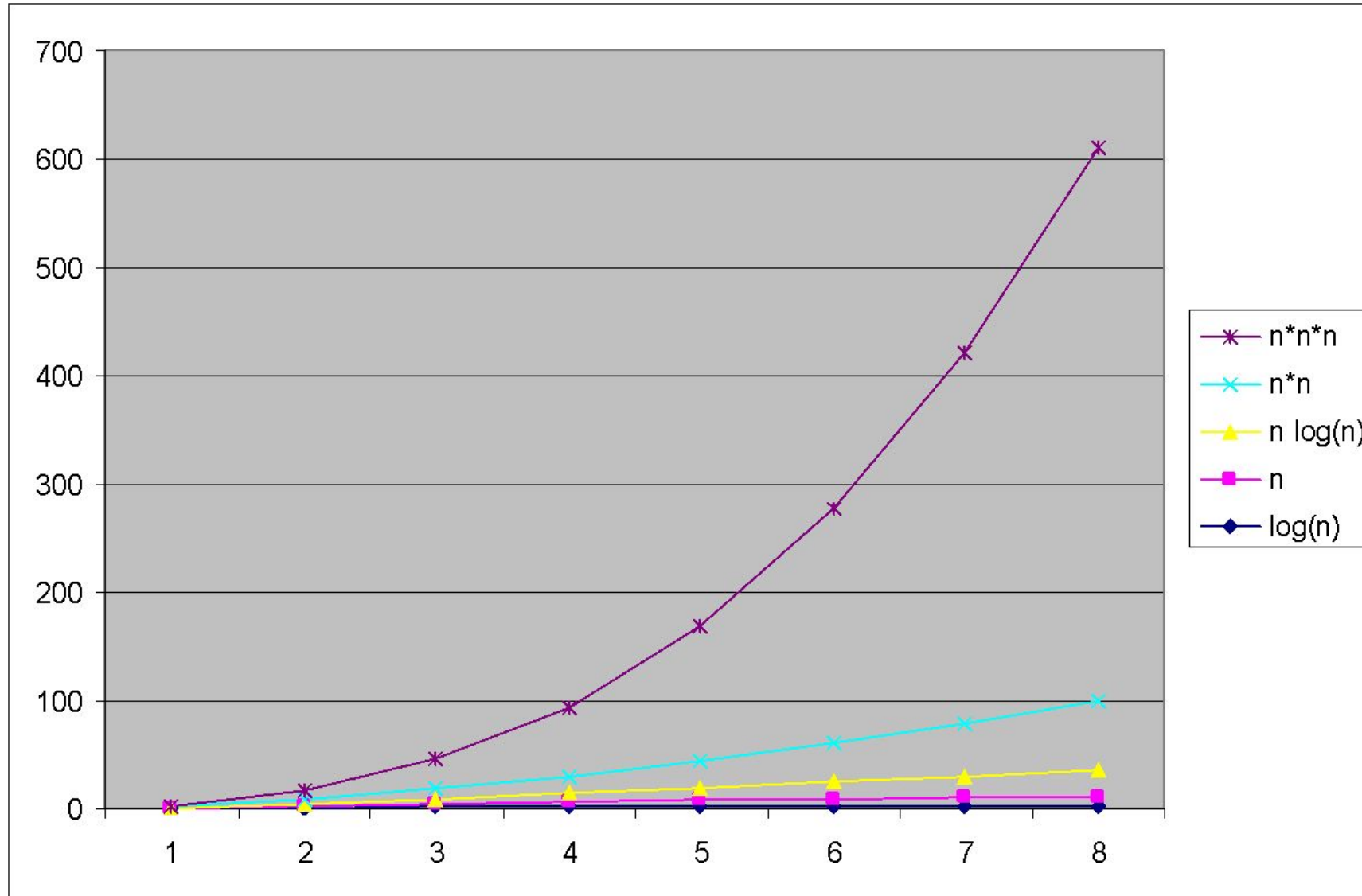
- Because, for smaller inputs, it is difficult to distinguish inefficient algorithms vs. efficient ones.
- For example, if the number of basic operations of two algorithms to solve a particular problem are n and n^2 respectively, then
 - if $n = 2$, Basic operation will be executed 2 and 4 times respectively for algorithm1 and 2.
Not much difference!!!
 - On the other hand, if $n = 10000$, then it does makes a difference whether the number of times the basic operation is executed is n or n^2 .

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$	Exponential-growth functions
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$	
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$	
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9			
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}			
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}			
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}			

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Orders of growth:

- consider only the leading term of a formula
- ignore the constant coefficient.



1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

Design and Analysis of Algorithms

Best, Worst and Average case Analysis



- Algorithm efficiency depends on the input size n
- For some algorithms efficiency depends on type of input.

Example: Sequential Search

Problem: Given a list of n elements and a search key K , find an element equal to K , if any.

Algorithm: Scan the list and compare its successive elements with K until either a matching element is found (successful search) or the list is exhausted (unsuccessful search)

Given a sequential search problem of an input size of n ,
what kind of input would make the running time the longest?
How many key comparisons?

□ Worst case Efficiency

- Efficiency (# of times the basic operation will be executed) for the worst case input of size n .
- The algorithm runs the longest among all possible inputs of size n .

□ Best case

- Efficiency (# of times the basic operation will be executed) for the best case input of size n .
- The algorithm runs the fastest among all possible inputs of size n .

□ Average case:

- Efficiency (# of times the basic operation will be executed) for a typical/random input of size n .
- NOT the average of worst and best case

□ How to find the average case efficiency?

ALGORITHM SequentialSearch($A[0..n-1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n-1]$ and a search key K

//Output: Returns the index of the first element of A that matches K or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ and $A[i] \neq K$ do

$i \leftarrow i + 1$

if $i < n$ // $A[i] = K$

 return i

else

 return -1

□ Worst-Case: $C_{\text{worst}}(n) = n$

□ Best-Case: $C_{\text{best}}(n) = 1$

□ Average-Case

from $(n+1)/2$ to $(n+1)$

Let '**p**' be the probability that key is found in the list

Assumption: All positions are equally probable

Case1: key is found in the list

$$C_{\text{avg, case1}}(n) = p * (1 + 2 + \dots + n) / n = p * (n + 1) / 2$$

Case2: key is not found in the list

$$C_{\text{avg, case2}}(n) = (1 - p) * (n)$$

$$C_{\text{avg}}(n) = p(n + 1) / 2 + (1 - p)(n)$$

Which of the following statements correctly differentiates *algorithm analysis* from *empirical program execution measurement*?

- a) Algorithm analysis focuses on CPU execution time, whereas program execution measurement focuses on memory usage
- b) Algorithm analysis evaluates efficiency in terms of input size and abstract operations, independent of hardware and programming language
- c) Program execution measurement is purely theoretical, while algorithm analysis is entirely experimental
- d) Algorithm analysis is applicable only to algorithms that use recursion

Answer

- b) Algorithm analysis evaluates efficiency in terms of input size and abstract operations, independent of hardware and programming language

An algorithm has low time complexity but performs poorly on a specific machine. According to the slides, this discrepancy is most likely due to:

- a) Poor algorithm design
- b) High space complexity
- c) External factors such as hardware speed and compiler quality
- d) Incorrect asymptotic analysis

Answer

- c) External factors such as hardware speed and compiler quality

In the space complexity formulation $S(P) = C + SP(I)$ which of the following contributes only to the variable space requirement?

- a) Instruction space
- b) Space for constants
- c) Recursive stack space
- d) Space for fixed-size variables

Answer

- c) Recursive stack space

Why is experimental study alone insufficient for comparing two algorithms?

- a) It cannot measure execution time accurately
- b) It ignores space complexity
- c) Results depend on selected inputs and execution environment
- d) It cannot be used for recursive algorithms

Answer

- c) Results depend on selected inputs and execution environment

Why does theoretical analysis model running time as a function of input size n rather than using actual time units (seconds, milliseconds)?

- a) Actual time units are inaccurate
- b) Time units vary with compiler quality only
- c) Actual running time depends on hardware and software environment
- d) Input size does not affect execution time

Answer

- c) Actual running time depends on hardware and software environment

In the order-of-growth analysis, multiplicative constants in the basic operation count $C(n)$ are ignored primarily because:

- a) They are too small to measure
- b) They depend on the programming language
- c) Growth rate dominates performance for large input sizes
- d) They affect only space complexity

Answer

- c) Growth rate dominates performance for large input sizes

For the problem of adding two $n \times n$ matrices, which choice correctly identifies the input size measure and the basic operation used in theoretical analysis?

- a) Input size: n^2 , Basic operation: multiplication
- b) Input size: n , Basic operation: addition
- c) Input size: number of elements, Basic operation: comparison
- d) Input size: n , Basic operation: assignment

Answer

- b) Input size: n , Basic operation: addition

An algorithm has a basic operation count

$$C(n) = \frac{1}{2} n(n - 1)$$

If the input size is doubled from n to $2n$, approximately how many times longer will the algorithm run?

- a) 2 times
- b) 4 times
- c) 8 times
- d) n times

Answer

- b) 4 times

Why is the basic operation count method preferred over the step count method for time complexity analysis?

- a) Step count gives incorrect results
- b) Step count ignores recursion
- c) Basic operation count focuses on the dominant cost contributing to runtime
- d) Basic operation count measures actual execution time

Answer

- b) Basic operation count focuses on the dominant cost contributing to runtime

Why is the order of growth of an algorithm more important than the exact number of basic operations for large input sizes?

- a) Because constants dominate execution time
- b) Because lower-order terms dominate for large inputs
- c) Because growth rate determines scalability as input size increases
- d) Because exact operation counts cannot be computed

Answer

- b) Because growth rate determines scalability as input size increases

Two algorithms have basic operation counts

$$C1(n) = 10n \quad \text{and} \quad C2(n) = n^2$$

For which range of input sizes is the difference in performance most significant?

- a) Very small n
- b) Moderate n
- c) Very large n
- d) Only when $n = 10$

Answer

- c) Very large n

Which of the following correctly orders the following functions by increasing order of growth?

- a) n , $\log n$, $n \log n$, n^2 , 2^n
- b) $\log n$, n , $n \log n$, n^2 , 2^n
- c) $\log n$, $n \log n$, n , n^2 , 2^n
- d) n , $n \log n$, $\log n$, n^2 , 2^n

Answer

- b) $\log n$, n , $n \log n$, n^2 , 2^n

For Sequential Search on a list of size n , which input causes the worst-case running time?

- a) Key is at the first position
- b) Key is at the middle of the list
- c) Key is at the last position
- d) Key is not present in the list

Answer

- b) Key is not present in the list

Given a Sequential Search where the probability that the key is present is p , and all positions are equally likely, the average-case number of comparisons is:

- a) $n/2$
- b) $(n+1)/2$
- c) $p(n+1)/2 + (1-p)n$
- d) $pn + (1-p)n/2$

Answer

- c) $p(n+1)/2 + (1-p)n$



THANK YOU

Vandana M L

Department of Computer Science & Engineering

vandanamd@pes.edu